Python Programming

Part of "Getting Started with Positron" workshop

Posit Software, PBC

hello-py.qmd — positron-workshop-project-sdss-2026

New | Open | Search | Python 3.14.3 (uv: positron-workshop-project-sdss-2026) | positron-workshop-project-sdss-...

EXPLORER

POSITRON-WORKSHOP-...
- .venv
- hello-py_files
- .gitignore U
- .python-version U
- hello-py.html U
- hello-py.qmd U
- hello-py.quarto... U
- hello-r.qmd U
- main.py U
- pyproject.toml U
- README.md U
- uv.lock U

OUTLINE
TIMELINE
QUARTO

Settings | hello-r.qmd U | hello-py.qmd U

Preview | Render on Save

hello-py.qmd > Meet the penguins > plot-penguins

```
25  The `penguins` data from the [**palmerpenguins**](https://allisonhorst.github.io/
    palmerpenguins "palmerpenguins R package") package contains size measurements for `
    {python} penguins.shape[0]` penguins from three species observed on three islands in
    the Palmer Archipelago, Antarctica.
26
27  The plot below shows the relationship between flipper and bill lengths of these
    penguins.
28
    ▷Run Cell | Run Above
29  ```{python}
30  #| label: plot-penguins
31  #| warning: false
32  #| echo: false
33  (
34      ggplot(penguins, aes(x="flipper_length_mm", y="bill_length_mm"))
35      + geom_point(aes(color="species", shape="species"))
36      + scale_color_manual(values=["#FF8C00", "#A020F0", "#008B8B"])
37      + labs(
38          title="Flipper and bill length",
39          subtitle="Dimensions for penguins at Palmer Station LTER",
40          x="Flipper length (mm)",
41          y="Bill length (mm)",
42          color="Penguin species",
43          shape="Penguin species",
44      )
45      + theme_minimal()
46  )
47  ```
48
```

CONSOLE | TERMINAL | PROBLEMS | OUTPUT | PORTS | DEBUG CONSOLE

```
...         x="Flipper length (mm)",
...         y="Bill length (mm)",
...         color="Penguin species",
...         shape="Penguin species",
...     )
...     + theme_minimal()
... )
```

R 4.5.2
Python 3.14.3 (...
CPU 0% MEM 195.36MB

Quarto: 1.10.3 Refreshing virtual environments

SESSION | CONNECTIONS | HELP | HISTORY | VIEWER

VARIABLES — 1.59GB

Python 3.14.3 (uv: positron-workshop-project-sdss-2026) — Filter

DATA

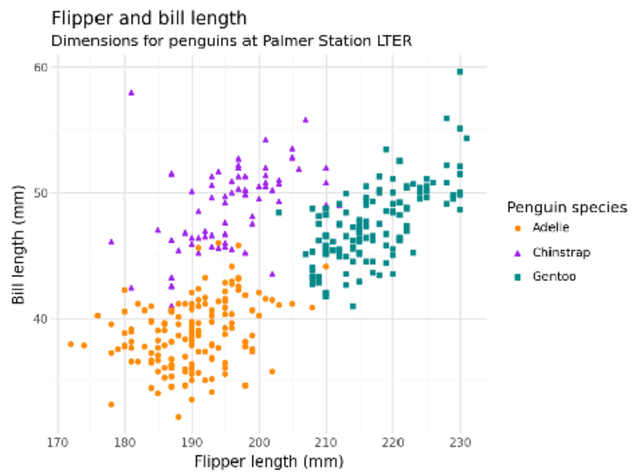
penguins	[344 rows x 8 columns] pandas.DataFrame	

FUNCTIONS

after_scale	after_scale(x)	function
after_stat	after_stat(x)	function
as_labeller	as_labeller(x: 'Optional[CanBeStripLabellingFunc]' = Non...	function
expand_limits	expand_limits(**kwargs)	function
ggsave	ggplot.save(self, filename: 'Optional[str | Path | BytesIO]' ...	function
label_both	label_both(label_info: 'strip_label_details', multi_line: 'bo...	function

PLOTS — Fit | plotnine (6.4inx4.8in)

Python 3.14.3 (uv: positron-workshop-project-sdss-2026) | hello-py.qmd | plotnine 1



main* | 0 0 | Ln 45, Col 22 | Spaces: 2 | UTF-8 | LF | Quarto | Prettier

A Python session in the Console

Package management with **uv**

Python packaging landscape

Python's packaging ecosystem has historically been fragmented:

- Multiple tools: `pip`, `virtualenv`, `venv`, `conda`, `poetry`, `pipenv`, etc.
- Multiple config files: `requirements.txt`, `setup.py`, `pyproject.toml`, etc.
- Version management often handled separately (`pyenv`)

`uv` is a modern tool that aims to unify these concerns with a fast, Rust-based implementation.

What is uv?

`uv` is a Python package and project manager developed by Astral.

Key features:

- Extremely fast (10-100x faster than pip)
- Manages Python versions
- Creates and manages virtual environments
- Installs packages
- Handles project dependencies via `pyproject.toml`
- Drop-in replacement for `pip` and `virtualenv`
- Directly supported by Positron and Reticulate

Installing uv

On MacOS/Linux:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

or with homebrew (recommended):

```
brew install uv
```

or with pip / pipx

```
pipx install uv
pip install uv
```

Verify installation

Once installed you should be able to run the following:

```
uv --version
```

```
uv 0.11.3 (Homebrew 2026-04-01 aarch64-apple-darwin)
```

As long as you have something higher than 0.9.* you should be fine.

Managing Python versions

uv can install and manage multiple Python versions:

```
uv python install 3.14
```

```
Python 3.14 is already installed
```

```
uv python pin 3.14
```

```
Pinned `.python-version` to `3.14`
```

The pinned version is stored in `~/.python-version` and will be used automatically.

Managing Python versions

uv can install and manage multiple Python versions:

```
uv python install 3.14
```

```
Python 3.14 is already installed
```

```
uv python pin 3.14
```

```
Pinned `.python-version` to `3.14`
```

The pinned version is stored in `~/.python-version` and will be used automatically.

Demo

Initializing a new project

- Open a new Positron window
- Create a Python project, with Git
- Create a new virtual environment with uv and launch the project
- Add packages to `pyproject.toml` with `uv add`

Activating environments in Positron with uv

Positron automatically detects virtual environments in your project directory. When you open a folder containing a `.venv` directory (created by uv), Positron will:

- Detect the environment and offer to use it
- Show the active Python interpreter in the status bar
- Use the environment for the Python console and when running scripts

If not automatically detected, you can manually select the interpreter via the Command Palette (`Cmd+Shift+P` / `Ctrl+Shift+P`) and searching for "Python: Select Interpreter".

Running Python code

Demo

Syncing an existing project

- Go to the workshop project
- In the Terminal, run `uv sync` to construct the venv and install all dependencies for the project.
- Activate the virtual environment
- Preview `hello-py.qmd`
- Run the invidual code cells in `hello-py.qmd` and inspect the variables pane

`uv sync`

Since the `.venv` folder is system specific (and large) it is not typically committed to git. Instead you will likely clone a repository that just has a `pyproject.toml` file.

Use `uv sync` to construct the venv and install all dependencies for the project.